以下为AI生成的图文笔记的内容

一、模型压缩策略 00:00

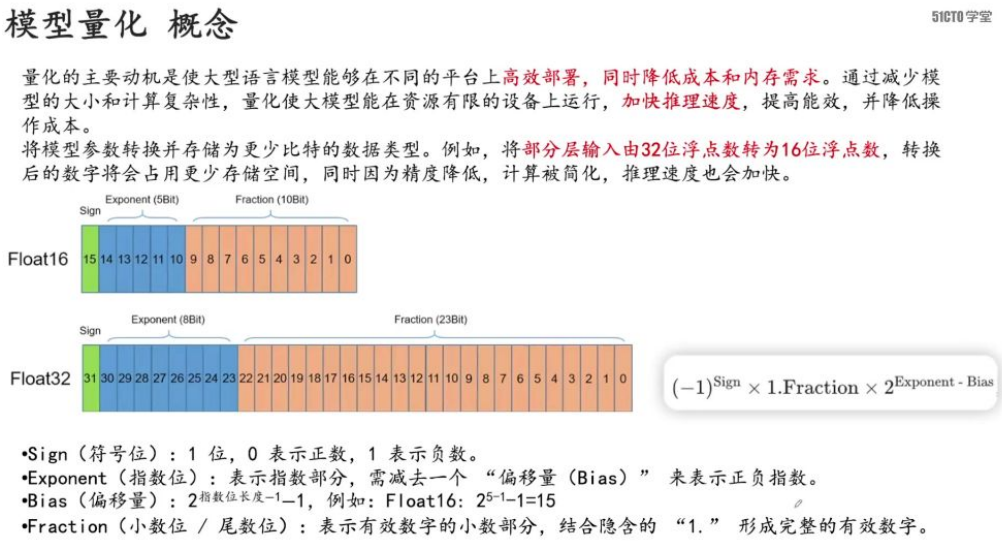
1. 模型压缩策略总结 00:16



- **三大技术路线**：量化、剪枝、蒸馏构成核心压缩手段
- **应用场景差异**：训练中(QAT)、微调中(QAF)、训练后(PTQ)三种量化时机
- **典型流程**：先压缩(量化/剪枝)后恢复(蒸馏)是常见组合方案

2. 模型量化 00:56

1）训练后量化 01:27

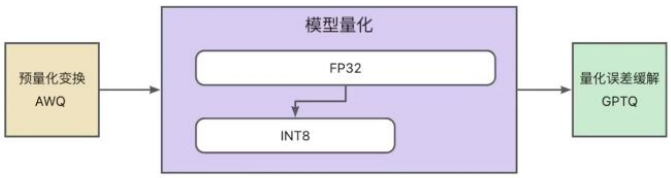


- **核心动机**：使大模型能在资源受限设备运行，降低3 × 内存需求
- **精度转换**：如FP32→INT8，浮点位数由$(-1)^S \times 1.F \times 2^{E-Bias}$公式决定
- **硬件优势**：16位浮点(5E+10F)比32位(8E+23F)减少50%存储空间

模型量化 训练后量化

51CTO学堂

训练后量化(Post Training Quantization,PTQ)：在模型训练完成后对其参数进行量化，只需要少量校准数据，适用于追求高易用性和缺乏训练资源的场景。



- 激活感知权重量化（AWQ）是一种仅权重量化方法，它通过离线分析激活分布来识别并采用等效缩放变换（Scaling）来保护模型中最显著的权重通道（Salient Weight Channels），从而在大幅压缩模型尺寸的同时最大限度地减少量化误差。
- GPTQ（Generative Pre-trained Transformer Quantization）单次训练后权重量化方法，它通过使用近似二阶信息（基于 Hessian 矩阵的补偿）来缓解量化误差，从而在将模型权重位宽降低至 3 或 4 位时，保持可忽略的准确性损失。

-
- **校准数据**：仅需少量样本(如1024条)即可完成参数校准
- **误差补偿**：GPTQ通过Hessian矩阵二阶近似补偿量化损失
- **保护机制**：AWQ保留0.1%-1%显著权重维持FP16精度

2）激活感知权重量化 02:26

- 量化 02:51

模型量化 AWQ-激活感知权重量化

51CTO学堂

Activation-aware Weight Quantization（激活感知权重量化）：在大型语言模型（LLM）的推理过程中，并非所有模型权重都对推理结果有着同等程度的影响。仅有0.1% - 1%的显著权重对模型输出精度起着关键作用。所以需要识别显著权重，并使其保持原来的高精度，而对其他权重进行低比特量化。

PPL：模型预测下一个词时的平均'困惑程度'，**数值越低，模型性能越好**。

PPL↓	FP16	RTN (w3-g128)	FP16% (based on act.)			FP16% (based on W)			FP16% (random)		
			0.1%	1%	3%	0.1%	1%	3%	0.1%	1%	3%
OPT-1.3B	14.62	119.00	25.03	16.91	16.68	108.71	98.55	98.08	119.76	109.38	61.49
OPT-6.7B	10.86	23.54	11.58	11.39	11.36	23.41	22.37	22.45	23.54	24.23	24.22
OPT-13B	10.13	46.04	10.51	10.43	10.42	46.07	48.96	54.49	44.87	42.00	39.71

- 随机挑选（random）：随机挑选显著权重的方法简单粗暴，就是随机选出0.1% - 1%的权重作为显著权重。
- 权重分布挑选（base on W）：对权重矩阵中的元素按绝对值大小由大到小进行排序，认为绝对值越大的元素越显著，然后选择前0.1% - 1%的元素作为显著权重。
- 激活值分布挑选（based on act）：这里的激活值是指与权重矩阵作矩阵乘法（matmul）运算的输入值。例如在自注意力机制中，计算相关结果时，对应的输入值就是相应权重矩阵的激活值。该方法按激活值绝对值大小由大到小排序，绝对值越大越显著，选择前0.1% - 1%的元素作为显著权重。

  -
  - **显著权重识别**：三种方法对比(随机/权重分布/激活分布)
  - **性能指标**：PPL值在OPT-13B模型从46.04降至10.42
  - **硬件优化**：混合精度计算效率差于统一INT4格式
- 过程 03:43

模型量化 AWQ量化-过程

51CTO学堂

步骤	机制	描述与来源
1. 识别显著权重	基于激活值的感知	AWQ 基于一个核心观察：**并非所有权重都同等重要**3。要确定哪些权重通道是显著的，应该**参考激活值的分布**，而非权重本身的分布。
2. 等效变换	数学等效缩放	AWQ 使用**等效变换**Y=XW=(X^)(W^)，将量化难度从难以处理的**激活值**分布转移到**权重**上。该变换在数学上不改变模型的输出结果。
3. 确定缩放因子	离线收集统计信息	AWQ 通过**离线收集激活值（X）的统计信息**（例如最大幅度），来确定最优的**每通道缩放因子 (S)**。
4. 保护与量化	放大显著权重	缩放因子 S 用于**放大**那些被判定为显著的权重通道 W^=S·W。这种放大操作旨在**保护**这些关键权重，使其在随后量化到低比特（如 INT4）时，能够**保留更多有效信息**。

- **四步流程**：识别→变换→缩放→量化形成完整链路
- **数学等效**：$Y = XW = (X / \alpha)(\alpha W)$保持输出不变
- **离线统计**：通过激活值最大幅度确定每通道缩放因子S

- 实现 03:52

模型压缩策略 AWQ量化-实现

51CTO学堂

AutoAWQ 是一个易于使用的用于 4 位量化模型的工具包。它通过实现激活感知权重量化（Activation - aware Weight Quantization，AWQ）算法对大语言模型（LLMs）进行量化。相比 FP16，AutoAWQ 能将模型推理速度提升 3 倍，同时将内存需求降低 3 倍。该工具包是在麻省理工学院（MIT）的原始工作基础上创建和改进而来的。目前，AutoAWQ 已被 vLLM 项目完全采用。

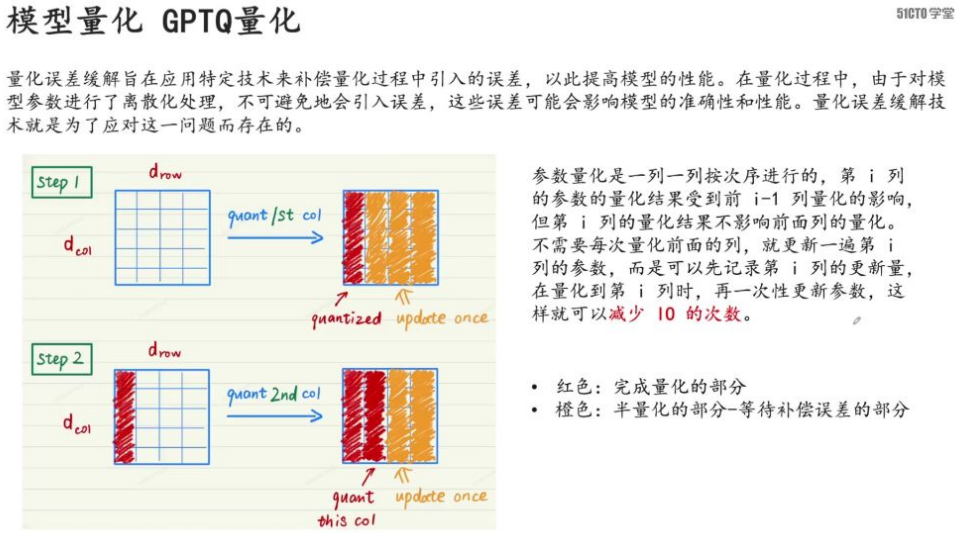
```
pip install autoawq
```

```python
# 导入AWQ量化所需的模型类和transformers的分词器类
from awq import AutoAWQForCausalLM
from transformers import AutoTokenizer
# 原始模型的路径（Hugging Face Hub上的Mistral-7B-Instruct-v0.2模型）
model_path = 'mistralai/Mistral-7B-Instruct-v0.2'
# 量化后模型的保存路径
quant_path = 'mistral-instruct-v0.2-awq'
# 量化配置参数：启用零点（zero_point），分组大小128，权重量化为4位，版本为GEMM
quant_config = { "zero_point": True, "q_group_size": 128, "w_bit": 4, "version": "GEMM" }
# 从预训练模型路径加载原始模型
model = AutoAWQForCausalLM.from_pretrained(model_path)
# 从预训练模型路径加载对应的分词器，trust_remote_code=True允许加载模型相关的自定义代码
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)
# 对模型进行量化，传入分词器和量化配置
model.quantize(tokenizer, quant_config=quant_config)
# 保存量化后的模型到指定路径
model.save_quantized(quant_path)
# 保存分词器到与量化模型相同的路径
tokenizer.save_pretrained(quant_path)
# 打印提示信息，表明模型已量化并保存
print(f'Model is quantized and saved at "{quant_path}"')
```

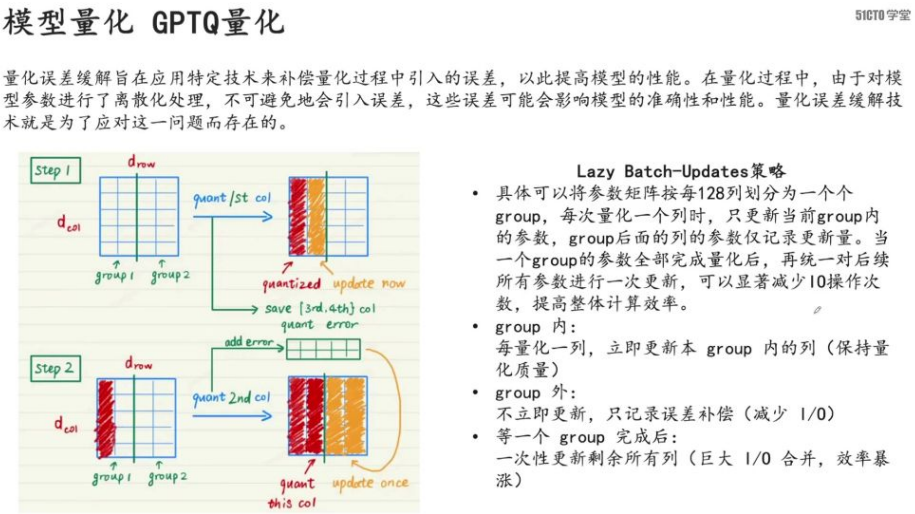
- **量化配置**：4bit权重+128分组大小+GEMM版本
- **典型加速**：相比FP16实现3倍推理速度提升
- **生态整合**：已被vLLM项目全面采用

3）GPTQ量化 04:06

- 量化误差缓解 04:54



-
- **串行补偿**：第i列量化误差补偿给i+1列
- **IO优化**：预量化策略减少50%以上存储访问
- Lazy Batch-Updates策略 05:01



-
- **分组量化**：每128列为单位进行误差累积
- **双阶段更新**：组内实时更新，组间延迟更新
- **效率提升**：合并IO操作实现效率"暴涨"

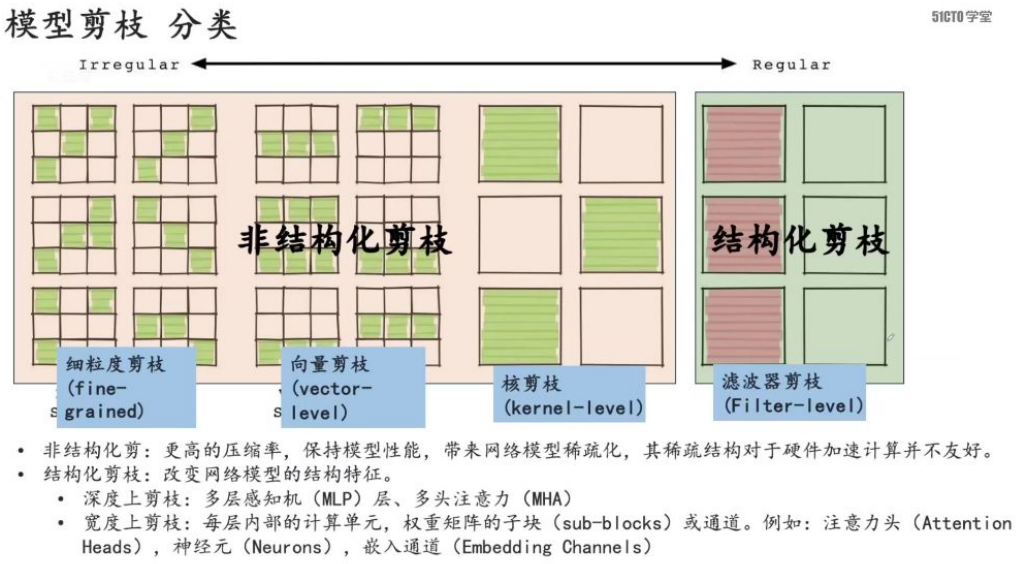
- GPTQ量化实践 05:48



- **校准数据**：使用C4数据集前1024条文本
- **量化准备**：需加载原始FP16模型和对应tokenizer
- **保存格式**：生成带"gptq-4bit"后缀的模型目录

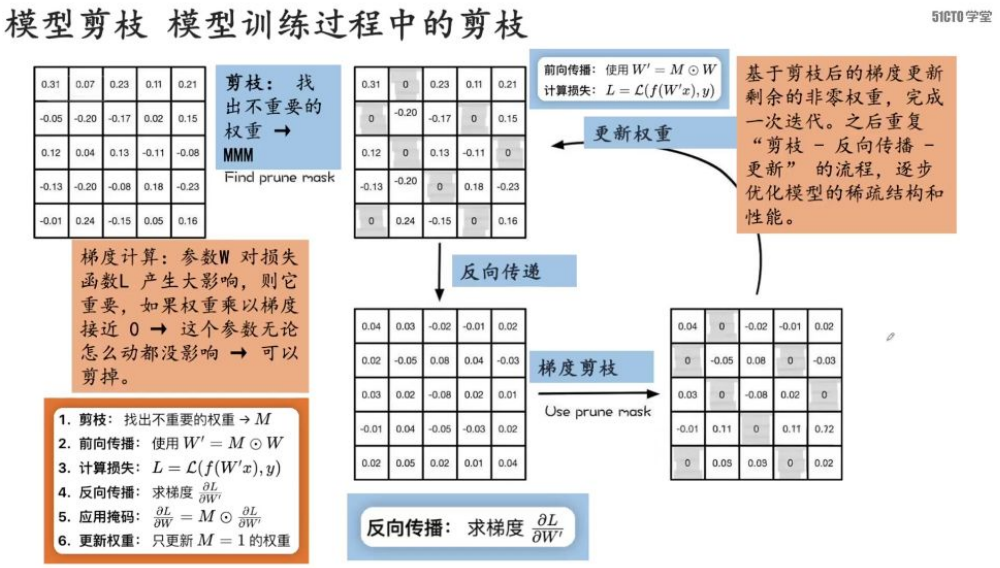
3. 模型剪枝 06:01

1）分类 06:06



- **结构化程度**：非结构化(细粒度/向量/核) vs 结构化(滤波器级)
- **硬件友好性**：结构化剪枝产生规整矩阵利于加速计算
- **稀疏控制**：DropOut(神经元级) vs DropConnect(连接级)

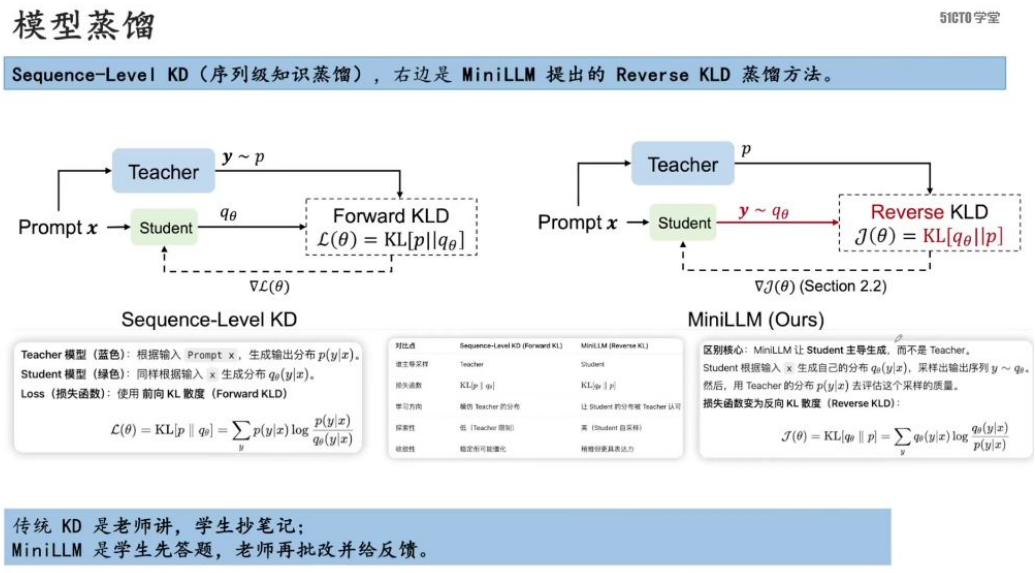
2）模型训练过程中的剪枝 06:24



-
- **重要性判定**：$\frac{\partial L}{\partial W} \times W$接近0的参数可剪
- **掩码应用**：前向传播使用$W' = M \odot W$进行稀疏计算
- **梯度更新**：仅更新$M = 1$的权重，保持50%-90%稀疏度

4. 模型蒸馏 06:53

1）知识蒸馏 07:23

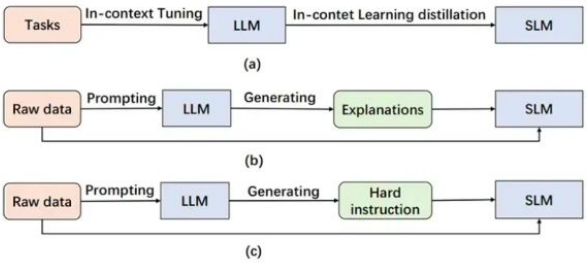


-
- **传统KD**：前向KL散度$\sum plog\frac{p}{q_\theta}$，老师主导
- **MiniLLM**：反向KL散度$\sum q_\theta log\frac{q_\theta}{p}$，学生主导
- **效果差异**：反向KL生成质量更高但训练稳定性稍差

2）应用场景 07:36

模型蒸馏

51CTO学堂



模式	主要学习对象	目标	输出样式	典型应用
(a) In-context Learning Distillation	上下文学习能力	从上下文中泛化	示例 + 任务	少样本学习、问答
(b) Chain-of-Thought Distillation	推理步骤	学习推理链路	推理文本	数学、逻辑推理、代码生成
(c) Instruction Following Distillation	指令执行	模仿执行方式	指令响应	对话系统、写作生成

-
- **上下文学习**：少样本示例+任务描述，用于QA场景
- **思维链蒸馏**：保留推理中间步骤，提升数学解题能力
- **指令跟随**：模仿人类指令响应模式，优化对话系统

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
模型压缩策略	分训练中、微调中、训练后三种场景，主流采用训练后量化（PTQ）	**Post-training quantization** 与训练中量化的区别	★★
量化技术	降低参数精度（如FP32→INT8）以减少内存占用和IO成本	**显著权重保护** vs 常规权重量化	★★★
AWQ量化	通过放大因子处理显著权重，保持量化后模型效果（PPL相近）	等效变换与缩放因子计算逻辑	★★★★
GPTQ量化	分组量化补偿误差（列间损失传递），减少IO操作次数	**128列矩阵分组策略** 与单次IO优化	★★★★
剪枝技术	移除不重要参数/连接，分非结构化（稀疏矩阵）和结构化	权重×梯度≈0 的判断标准	★★★
模型蒸馏	大模型（教师）指导小模型（学生），分笔记式与答题反馈式	**涌现能力** 的参数量阈值	★★★★
蒸馏应用场景	少样本问答（上下文学习）、数学推理（思维链）、指令执行（对话生成）	三类场景的**任务适配性**对比	★★★